INTEGRATION + E2E + CONTRACT ТЕСТУВАННЯ

Від зеленої брехні моку до can-i-deploy: testcontainers, supertest, Pact + Broker

Заняття №16

Node.js Pro

Лектор
Кирило Орлов

r_d

СТЕК: JEST · TESTCONTAINERS · SUPERTEST · PACT · BROKER

Перша половина — довіра всередині сервісу	Друга половина — довіра між шарами й командами
крок 1 · юніт із моком: межа з L#10	крок 5 · supertest: E2E повного Nest
крок 2 · testcontainers: справжній Postgres	крок 6 · Pact: contract + broker + can-i-deploy
крок 3 · ціна ізоляції: контейнер vs ROLLBACK vs TRUNCATE	крок 7 · maxWorkers: паралельність і бюджет CI
крок 4 · builders: дані без стіни фікстур	Зелений тест ≠ правда. Правда коштує часу

НАСКРІЗНА НИТКА

Кожен щабель купує довіру за час: юніт — мілісекунди й віра; контейнер — секунда й факт; E2E — вся система; контракт — окремий цикл і гарантія між командами. Код лекції — драбинка кроків 1-7, кожен крок = одна команда.

ПЛАН ЗАНЯТТЯ



- Зелена брехня юніта і справжній Postgres у testcontainers
- Ціна ізоляції: контейнер-на-тест vs ROLLBACK vs TRUNCATE; builders
- E2E повного Nest через supertest: усі шари справжні
- Contract testing: Pact consumer → provider → broker → can-i-deploy
- Паралельність, межі теми і ДЗ для курсового (7 балів)

ПРОБЛЕМА: ЗЕЛЕНИЙ ЮНІТ-ТЕСТ УМІЄ БРЕХАТИ

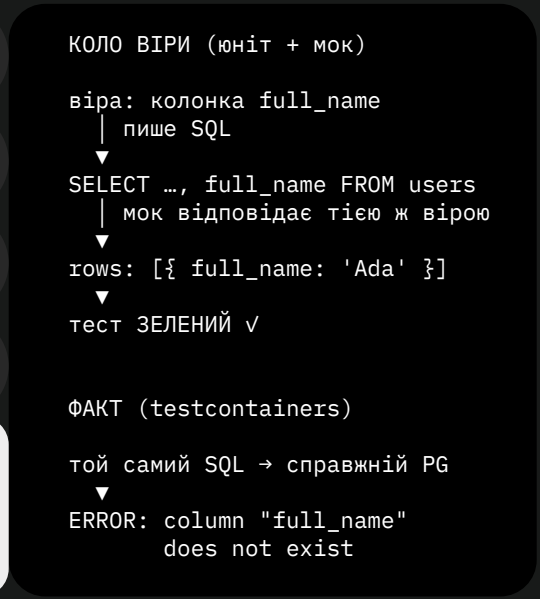
на L#10 ми зупинились на юнітах: швидкі, дешеві — і вірять мокам

розробник вірить у колонку full_name — у схемі вона name

мок повертає те, у що вірить розробник, — ту саму full_name

обидва боки написала одна людина: тест перевіряє самоузгодженість

SQL, який жодного разу не бачила жодна БД, — зелений у CI місяцями. Потрібен щабель, де відповідає не мок, а справжній Postgres.



КРОК 1 · ЮНІТ ІЗ МОКОМ: ЗЕЛЕНИЙ НА ЗЛАМАНОМУ SQL

```
test('мок дзеркалить помилкову віру розробника', async () =>
{
  const db = {
    query: jest.fn().mockResolvedValue({
      rows: [{ id: 1, email: 'ada@example.com', full_name:
'Ada' }],
    }),                        // ← відповідь ДИКТУЄМО самі
  };

  const repo = new UsersRepoBroken(db);  // SQL усередині
ЗЛАМАНИЙ
  const user = await repo.findByEmail('ada@example.com');

  expect(user).toEqual({ id: 1, email: 'ada@example.com',
name: 'Ada' });
  expect(db.query).toHaveBeenCalledWith(
    'SELECT id, email, full_name FROM users WHERE email =
$1',
    ['ada@example.com'],       // колонки full_name НЕ ІСНУЄ
  );
});  // Tests: 1 passed · 0.089 s — БД цей SQL не бачила
```

Мок дзеркалить віру розробника — обидва боки однакові

1 · jest.fn().mockResolvedValue — відповідь диктуємо самі

2 · expect(...) — і мапінг «працює», і виклик «правильний»

3 · full_name не існує в БД — але тут БД немає

npm run step1 → Tests: 1 passed · Time: 0.089 s

урок: мок звіряє код із вірою, а не з БД

«КРАШ-ТЕСТ»: РІВНІ ТЕСТУВАННЯ ЯК АНАЛОГІЯ

ЧАСТИНА ТЕСТІВ	АНАЛОГІЯ (КРАШ-ТЕСТ)
• юніт із моком • інтеграційний із testcontainers • E2E через supertest • contract через Pact	• манекен і намальована стіна: швидко, дешево, сотні разів • справжня бетонна стіна на полігоні • виїзд зібраного авто: гальма, подушки, електроніка разом • сертифікація фаркопа: ваш гак сумісний із ЧУЖИМ причепом

Де аналогія ламається: полігон — не дорога. У проді інші дані, навантаження й мережа, тому верх драбинки — не ще один тест, а спостережуваність (попереду в L#29). І краш-тест руйнує авто, а тести мають лишати БД чистою — про це ізоляція (крок 3).



ДРАБИНКА ДОВІРИ: ЩО КУПУЄМО І СКІЛЬКИ ПЛАТИМО

нижчий щабель — швидше й дешевше, але менше правди

правило: тест живе на НАЙНИЖЧОМУ щаблі, де видно факт

чиста логіка — юніт; SQL, схема, constraints — інтеграція

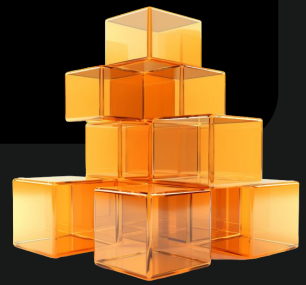
маршрути, пайпи, статуси — E2E; межі сервісів — контракт

типова помилка: тягнути все в E2E — повільно і хитко

піраміда з L#10 не зникла — ми добудовуємо її верх

ЩАБЕЛЬ	ВІДПОВІДАЄ	ЦІНА
контракт	чужа команда	окремий цикл
E2E	вся система	секунди
інтеграція	справжній PG	~1.3 с старт
юніт + мок	сам розробник	мілісекунди

довіра росте ↑ · швидкість падає ↓





```
// testkit/pg-container.ts — один виклик = один справжній Postgres

import { PostgreSqlContainer } from '@testcontainers/postgresql';

export async function startPg(label = ''): Promise<PgHandle> {
  const t0 = Date.now();
  const container = await new PostgreSqlContainer('postgres:16-alpine')
    .start();                       // Docker: образ → контейнер → готово
  const startupMs = Date.now() - t0;

  const uri = container.getConnectionUri();  // ЕФЕМЕРНИЙ хост-порт
  const pool = new Pool({ connectionString: uri });
  await pool.query(SCHEMA_SQL);               // свіжа схема щоразу

  return { container, pool, startupMs, uri,
    async stop() { await pool.end(); await container.stop(); } };
}
// порт обирає testcontainers — воркери не б'ються за 5432
// ryuk-контейнер прибере все, навіть якщо тести впали
```

```
// крок 2 — той самий зламаний SQL проти СПРАВЖНЬОГО Postgres

let pg: PgHandle;
beforeAll(async () => { pg = await startPg('(крок 2)'); });
afterAll(async () => { await pg.stop(); });

test('зламаний репозиторій: Postgres відповідає помилкою', async () => {
  const repo = new UsersRepoBroken(pg.pool);   // ТОЙ САМИЙ клас
  await expect(repo.findByEmail('ada@example.com')).rejects.toThrow(
    'column "full_name" does not exist',
  );
});

// npm run step2:
//   [testkit] postgres:16-alpine (крок 2) готовий за 1293 ms
//     → postgres://test:test@localhost:55001/test
//   Tests: 3 passed — справжня БД впіймала те, що мок пропустив
```



КРОК 3 · ІЗОЛЯЦІЯ ТЕСТІВ: ЦІНА ПИТАННЯ

A · контейнер на КОЖЕН тест: 2 тести — 2480 ms

B · один контейнер, BEGIN … ROLLBACK: ті самі 2 тести — 8 ms

C · один контейнер, TRUNCATE … RESTART IDENTITY: 12 ms



Дефолт для suite — B або C: мілісекунди і чиста БД між тестами. A лишається для міграцій і DDL, де потрібен незайманий інстанс. Цифри — реальний npm run step3 (Tests: 6 passed); наживо побачимо в демо B.



```
// стратегія B: кожен тест — у транзакції, яка НЕ комітиться

beforeEach(async () => {
  client = await pg.pool.connect();   // одна сесія на тест
  await client.query('BEGIN');
});
afterEach(async () => {
  await client.query('ROLLBACK');     // все, що тест написав, зникло
  client.release();
});

test('тест 1 бачить свої дані всередині транзакції', async () => {
  const repo = new UsersRepo(client); // репозиторій приймає Queryable
  await repo.create('b@example.com', 'B');
  expect(await repo.count()).toBe(1);
});
test('тест 2: ROLLBACK стер усе — таблиця порожня', async () => {
  expect(await new UsersRepo(client).count()).toBe(0);
});  // пастка: код із власним COMMIT ламає цю стратегію
```

КРОК 4 · TEST DATA BUILDERS: ДАНІ БЕЗ СТІНИ ФІКСТУР(wall of fixtures)

Wall of fixtures

15 рядків arrange у кожному тесті: що з них ВАЖЛИВЕ для перевірки — не видно

aUser()

валідні дефолти + лічильник: user-1@example.com, user-2@... — унікальність безкоштовна

insertVia(repo)

arrange одним рядком: await aUser().withEmail('dup@example.com').insertVia(repo)

Чому це важливо

У тесті звучить ЛИШЕ поле під перевірку — решту ховає builder. Тест читається як специфікація: withEmail('dup@...') означає «цей тест про дублікат email». Ланцюжок with*() росте разом із доменом курсового. npm run step4 → Tests: 2 passed.

ЮНІТ ПРОТИ ІНТЕГРАЦІЙНОГО: ЧЕСНА ТАБЛИЦЯ

Аспект	**юніт + мок**	**інтеграція (testcontainers)**
Швидкість	мілісекунди	~1.3 с старт + мс на тест
Хто відповідає	мок — ваша ж віра	справжній Postgres
Що ловить	логіку, мапінг, розгалуження	SQL, схему, constraints
Що потрібно	нічого	Docker — локально і в CI
Коли бреше	щойно розійшовся з БД	коли розійшлися СЕРВІСИ → контракт

КРОК 5 · SUPERTEST: E2E ПОВНОГО ЗАСТОСУНКУ

01. createTestingModule + AppModule — повний DI-граф, без підмін

02. configureApp() — той самий ValidationPipe, що й у проді

03. supertest б ′ є в app.getHttpServer() — і сам піднімає його на ефемерному порту

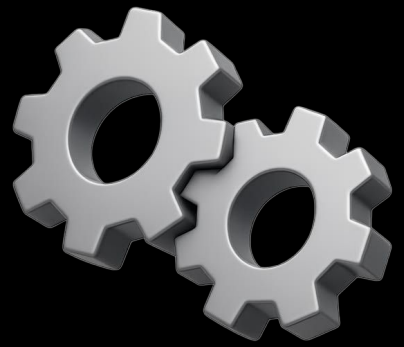
04. БД — testcontainer: DATABASE_URL підсунули через env

05. статуси справжні: 201 · 200 · 404 · 400 · 500 — жодного моку

ШЛЯХ ОДНОГО ЗАПИТУ: ЩО НАСПРАВДІ ПЕРЕВІРЯЄ E2E

```
POST /users { email: 'dup@example.com' }      — ВДРУГЕ
     ↓
ValidationPipe     кривий email? → 400 і СТОП
     ↓
UsersController  →  UsersService
     ↓
UsersRepo:  INSERT INTO users (email, name) …
     ↓
Postgres:  duplicate key … unique constraint   (23505)
     ↓
обробника конфліктів немає → Nest відповідає 500

 ── кожен рядок — СПРАВЖНІЙ шар, жодного моку ──
 юніт цей 500 не покаже: він не знає про constraint
```



КРОК 5: 4 ТЕСТИ · 6 ЗАПИТІВ — ХТО ВИРІШУЄ СТАТУС

Запит	Хто вирішує	Статус
POST /users (валідний)	контролер + БД: INSERT … RETURNING id	201
GET /users/:id (існує)	сервіс → репозиторій → рядок із БД	200
GET /users/999999	NotFoundException у сервісі	404
POST із кривим email	ValidationPipe — ДО сервісу й БД	400
POST із дубль-email	Postgres unique 23505, обробника нема	500

```
// крок 5 — happy path і дубль email: усі шари справжні

test('POST /users → 201, GET /users/:id → 200', async () => {
  const created = await request(app.getHttpServer())
    .post('/users')
    .send({ email: 'ada@example.com', name: 'Ada' })
    .expect(201);

  const fetched = await request(app.getHttpServer())
    .get(`/users/${created.body.id}`)
    .expect(200);
  expect(fetched.body).toEqual(created.body);
});

test('дубль email → помилка з БД, а не з моку', async () => {
  await request(app.getHttpServer()).post('/users')
    .send({ email: 'dup@example.com', name: 'Перший' }).expect(201);
  await request(app.getHttpServer()).post('/users')
    .send({ email: 'dup@example.com', name: 'Другий' }).expect(500);
});   // npm run step5 → Tests: 4 passed
```



КРОК 6 · PACT: КОНТРАКТ МІЖ КОМАНДАМИ

E2E живе в ОДНОМУ репо — розрив між сервісами він не ловить

OpenAPI з L#9 — обіцянка; Pact — її виконувана перевірка

consumer-тест описує очікування → файл pacts/web-app-users-api.json

given('user 42 exists') — provider state: стан БД, у якому обіцянка діє

provider verification: СПРАВЖНІЙ users-api + stateHandlers сідять БД

matchers like(42) фіксують ФОРМУ, не значення — контракт не крихкий

«РОЗЕТКА І ВИЛКА»: КОНТРАКТ ЯК АНАЛОГІЯ

ЧАСТИНА PACT

- контракт pacts/*.json
- consumer-тест із мок-сервером
- provider verification
- Pact Broker + can-i-deploy
- де аналогія ламається

АНАЛОГІЯ (РОЗЕТКА)

- стандарт розетки, узгоджений на папері
- виробник вилки міряє СВОЮ вилку калібром
- завод розеток міряє СВОЮ розетку тим самим калібром
- реєстр: кого з ким перевірили — тим і можна вмикатись
- калібр міряє форму, не поведінку: перейменували суть поля без зміни типу — контракт мовчить

```
// крок 6a — consumer: web-app описує, чого чекає від users-api

const provider = new PactV3({ consumer: 'web-app',
  provider: 'users-api', dir: path.resolve('pacts') });

provider
  .given('user 42 exists')                    // provider state
  .uponReceiving('запит користувача 42')
  .withRequest({ method: 'GET', path: '/users/42' })
  .willRespondWith({ status: 200,
    body: { id: like(42), email: like('ada@example.com'),
            name: like('Ada') } });           // форма, не значення

await provider.executeTest(async (mockServer) => {
  const res = await fetch(`${mockServer.url}/users/42`);
  expect(res.status).toBe(200);               // тест б'є в МОК-СЕРВЕР
});    // артефакт: pacts/web-app-users-api.json — це і Є контракт
```

БРОКЕР І CAN-I-DEPLOY: КОНТРАКТ ЖИВЕ НЕ В РЕПО

publish: PUT .../pacts/.../version/1.0.0 → 201 (брокер на 21620)

до верифікації: can-i-deploy → {"deployable":null, unknown:1} — деплой ЗАБОРОНЕНО

провайдер верифікує З БРОКЕРА, публікує результат; тег prod

після: {"deployable":true} — «All required verification results are published and successful». Це і є гейт у CI: перед деплоєм пайплайн ПИТАЄ брокер. Ламаєш контракт — дізнаєшся на CI, а не від чужого пейджера. Wiring у GitHub Actions — L#32.



КРОК 7 · MAXWORKERS: ПАРАЛЕЛЬНІСТЬ МНОЖИТЬ КОНТЕЙНЕРИ

--maxWorkers=1

3 контейнерні файли послідовно: Time 3.971 s · контейнери 981 / 988 / 1119 ms

--maxWorkers=3

ті самі файли: Time 2.381 s · контейнери 1017 / 1625 / 1820 ms — докер ділить CPU та I/O

бюджет CI

кожен воркер = свій контейнер: RAM і CPU ростуть лінійно, на слабкому раннері виграш тане

Чому важливо

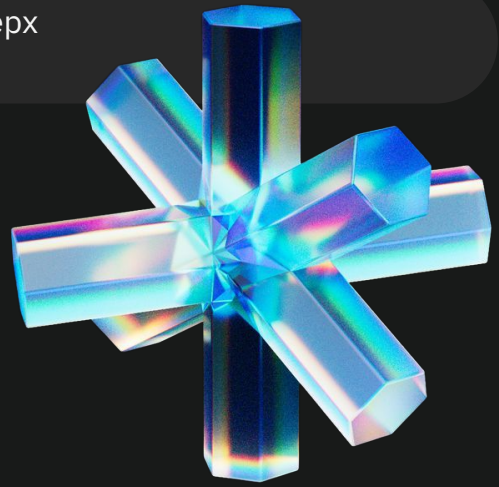
Паралельність не безкоштовна: сумарно швидше, але КОЖЕН контейнер стартує довше. У ДЗ дефолт maxWorkers: 1 — свідомо: спершу стабільно зелений suite, потім швидкість. Заміряй на СВОЄМУ CI, перш ніж крутити.

ЩО СВІДОМО ЛИШАЄМО ЗА ДВЕРИМА

Генеративні техніки: property-based (fast-check) — інваріанти замість прикладів; mutation testing (Stryker) — сила ваших тестів. Сьогодні лише імена — обидві лягають ПОВЕРХ цієї драбинки

Redis у testcontainers і тестування черг — глибоко в L#23. Повний CI-wiring contract-джоби (publish → verify → can-i-deploy у GitHub Actions) — L#32; сьогодні гейт показуємо локально через compose

⚠ Тести не скасовують спостережуваність: у проді інші дані й навантаження. Верх драбинки — логи, метрики й алерти, попереду в L#29, а не ще один шар тестів



ПІДСУМКИ + МІСТ ДО L#17 + ДЗ

ФАКТИ

мок — віра;
testcontainers дає відповідь справжнього Postgres за ~1.3 с

КОМАНДИ

Pact + брокер: розрив контракту видно в CI, а не в проді

ДАЛІ

L#17 GraphQL — інша форма API, та сама драбинка тестів · L#18 Realtime

ДЗ · 7 балів

ДЗ 7 балів (PROJECT MEDIUM): testcontainers-suite + ізоляція + builders + E2E supertest + Pact-контракт (OpenAPI з ДЗ#9) + broker + can-i-deploy у CI · дедлайн 22.09

ШАРИ

supertest ганяє ПОВНИЙ Nest: DI, пайпи, статуси, constraints — без моків

Q&A

???



YOUR PLACE
TO GROW